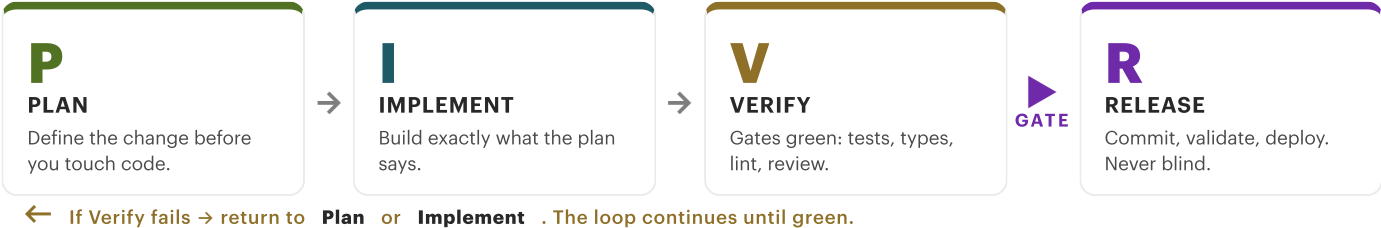


Industry Solutions PIV Loop Cheatsheet.

Plan → Implement → Verify, on loop.
Then Release.

! **How to use this sheet.** Every session begins with `/prime`. Then run the **PIV loop** — Plan, Implement, Verify — until the gate is green. Only then do you Release. Page 2 shows which loop to run; Page 3 helps you choose.



P · 15-30 MIN

PLAN

Build shared understanding **before** writing code.

<code>/prime</code>	Load project context. Every session.
<code>/plan:task</code>	Plan any single change — the minimum for any work.
<code>/plan:prd</code>	Define WHAT + WHY for a brand-new feature.
<code>/plan:spec</code>	Convert PRD into a technical spec.
<code>/plan:api</code>	Design the REST contract before coding endpoints.
<code>/plan:feature</code>	Scaffold the feature folder once the plan is approved.
<code>/plan:service</code>	Adding Redis, Supabase or other infra? Start here.

RULE Never implement without a plan.

I · 30 MIN - HOURS

IMPLEMENT

Build **exactly** what the plan says — nothing more.

<code>/implement:execute</code>	Build the feature or function end-to-end.
<code>/implement:migrate</code>	Guided DB migration: generate → review → apply.
<code>/implement:refactor</code>	Restructure code without changing behaviour.

RULE If you find yourself wandering off the plan, stop and re-plan.

V · 5-20 MIN

VERIFY

All gates green **before** you commit.

<code>/verify:run</code>	Tests + types + lint. After every change.
<code>/verify:code-review-fix</code>	Auto-fix obvious issues. Run <i>before</i> review.
<code>/verify:code</code>	Review your own diff. Always before commit.
<code>/verify:debug</code>	When <code>/verify:run</code> fails — diagnose.
<code>/verify:pr</code>	Full branch review before merging to main.
<code>/verify:security</code>	OWASP audit before production deploy.

RULE If it isn't green, it isn't done.

R · 5-60 MIN

RELEASE

Ship to production **safely**, never blindly.

<code>/release:commit</code>	Create a git commit with a sensible message.
<code>/release:docs</code>	Update README + architecture docs.
<code>/release:validate</code>	Full validation: tests + Docker + types.
<code>/release:env</code>	Audit env vars + secrets before prod.
<code>/release:deploy</code>	Pre-deploy checklist — final go / no-go.
<code>/release:rollback</code>	Reverse the last deploy if things go sideways.

RULE Friday deploys are weekend deploys in disguise.

Loop the loop. **Plan, Implement, Verify.**
Release is the reward — not the reflex.

From scratch, or **add a function?**

Same loop. Different scope.
Pick before you start.

SCENARIO 1 · FROM SCRATCH

Brand-new feature, brand-new folder.

Use when no folder yet exists for this domain. Bigger Plan, full scaffold, broad test coverage. Worth the time when the feature touches multiple systems or needs PRD alignment.

1	/prime	load context
2	/plan:prd	what + why
3	/plan:spec	API + schema + logic
4	/plan:api	REST contract
5	/plan:feature	scaffold the folder
6	/implement:execute	build everything
7	/verify:run → :code-review-fix → :code	gate green
8	/github:draft → /github:sync	team feedback
9	/release:commit → :deploy → /monitor:health	ship + watch

```
// What gets created
app/items/
├ models.py           // SQLAlchemy + migration
├ schemas.py          // Pydantic in/out
├ routes.py           // FastAPI endpoints
├ service.py          // business logic
├ repository.py        // DB queries
├ exceptions.py        constants.py
├ types.py            README.md
└ tests/test_routes.py test_service.py test_repository.py
app/main.py           // updated: wire router
alembic/versions/abc_add_items_table.py
```

END-TO-END

3 – 5 hrs

SCENARIO 2 · ADD A FUNCTION

Feature exists. You're adding to it.

Use when the folder is already in the codebase and you're adding one endpoint, query, or behaviour. Skip the PRD, skip the scaffold. Tight loop, fast finish.

1	/prime	load context
2	/plan:task	scope this change only
3	/implement:execute	add fn + tests
4	/implement:migrate	only if DB field changes
5	/verify:run	tests + types + lint
6	/verify:code-review-fix	auto-fix obvious
7	/verify:code	review the diff
8	/release:commit	done

```
// What changes (minimal)
app/items/
├ models.py           // maybe: new field
├ schemas.py          // maybe: new shape
├ routes.py           // + new endpoint
├ service.py          // + new method
├ repository.py        // + new query
└ tests/test_routes.py // + new test
// app/main.py – no change (router already wired)
// alembic – only if you added a DB field
```

END-TO-END

~ 50 min

ASPECT	SCENARIO 1 · FROM SCRATCH	SCENARIO 2 · ADD FUNCTION
Planning	/plan:prd + /plan:spec + /plan:api	/plan:task only
Scaffolding	/plan:feature creates folder	None — folder exists
Files	10 + new files	Update 3 – 4 existing
Database	New table + migration	Only if adding a field
Tests	Cover all paths (10 +)	2 – 3 tests for new fn
Time	3 – 5 hrs	30 – 50 min
Complexity	High — many decisions	Low — clear direction

Same PIV loop. **Different scope.**
Match the planning to the size of the change.

Which path? **And what does it look like?**Start here. **What are you building?**

If you can answer the question on the left, the commands on the right are the floor — not the ceiling.

Brand-new feature — no folder yet?→ **Scenario 1 · From scratch**

```
/plan:prd → /plan:spec → /plan:api → /plan:feature
```

≈ 3 – 5 HRS

Adding one endpoint or function to an existing feature?→ **Scenario 2 · Add function**

```
/plan:task → /implement:execute
```

≈ 50 MIN

Adding a new field to an existing model?→ **Scenario 2 + migration**

```
/plan:task → /implement:migrate → /implement:execute
```

≈ 1 HR

Fixing a bug in existing code?→ **Scenario 2 · Minimal**

```
/plan:task → /implement:execute → /verify:run
```

≈ 20 – 40 MIN

EXAMPLE · FROM SCRATCH**Reviews feature — users rate items.**

"Add a Reviews feature where users rate items."

```
1 /plan:prd    why & impact
2 /plan:spec
POST /reviews · GET /items/{id}/reviews
3 /plan:api    REST contract
4 /plan:feature creates app/reviews/
5 /implement:execute
6 /verify:run → /verify:code
```

EXAMPLE · ADD FUNCTION**Add DELETE endpoint to items.**

"Add DELETE /api/items/{id} to items feature."

```
1 /plan:task    scope the delete
2 /implement:execute
routes + service + repo + test
3 /verify:run
4 /verify:code
5 /release:commit
```

EXAMPLE · ADD FIELD + MIGRATION**Add 'category' field to items.**

"Add a category field on the items model."

```
1 /plan:task    migration + schema + tests
2 /implement:migrate generate & apply
3 /implement:execute
model + schema + tests
4 /verify:run → /verify:code
```

USE SCENARIO 1 · FROM SCRATCH WHEN...

- ☐ The feature folder doesn't exist yet.
- ☐ The work needs PRD alignment with another team.
- ☐ It touches auth, payment, or another sensitive system.
- ☐ The API contract needs careful design discussion.
- ☐ Multiple endpoints and complex logic are involved.

USE SCENARIO 2 · ADD FUNCTION WHEN...

- ☐ The feature folder already exists.
- ☐ You're adding one endpoint or one function.
- ☐ No PRD is needed — scope is clear.
- ☐ No cross-team discussion is required.
- ☐ The whole thing fits in `/plan:task` .

DON'T DO THIS

- ✗ Run `/plan:prd` for adding a delete endpoint.
- ✗ Run `/plan:feature` when the folder already exists.
- ✗ Mix a refactor and a feature in one change.
- ✗ Skip `/plan:task` "just this once".
- ✗ Commit on a red `/verify:run` .
- ✗ Deploy without `/release:env` — silent secrets bugs are the worst.

Right-size the loop. **The plan is cheap.** The cleanup isn't.

Every command, **in order, by phase.**

Numbered = typical order.
Dashed = optional / situational.

How to read this page.

1 typical step in the phase

optional · run when the situation calls for it

P phase tag

P PLAN · build understanding first
Every session starts here. Match the planning depth to the scope:
`/plan:task` is the floor; `/plan:prd` is for net-new features.

0	<code>/prime</code>	Load full project context. Stack, conventions, recent activity.
1	<code>/plan:task</code>	Plan any single change. The minimum for every piece of work.
2	<code>/plan:prd</code>	Write the product requirements doc — WHAT + WHY for a new feature.
3	<code>/plan:spec</code>	Convert PRD into a technical spec — API + schema + logic.
4	<code>/plan:api</code>	Design the REST contract — paths, verbs, payloads, errors.
5	<code>/plan:feature</code>	Scaffold the feature folder once the plan is approved.
	<code>/plan:agent</code>	Scaffold an AI agent feature (Pydantic AI).
	<code>/plan:service</code>	Add new infra (Redis, Supabase, queue) before wiring it in.

I IMPLEMENT · build the plan

1	<code>/implement:execute</code>	Build end-to-end. Routes, services, repos, tests.
	<code>/implement:migrate</code>	Run first when schema changes.
	<code>/implement:refactor</code>	Restructure without changing behaviour.

V VERIFY · gates green before commit
Fast checks every time. Specialised checks when the situation calls.

FAST · EVERY CHANGE

1	<code>/verify:run</code>	Tests + types + lint. After every change.
2	<code>/verify:code-review-fix</code>	Auto-fix obvious issues. Run <i>before</i> review.
3	<code>/verify:code</code>	Review your own diff. Always before commit.

SPECIALISED · WHEN THE SITUATION CALLS

	<code>/verify:debug</code>		<code>/verify:execution-report</code>
	Diagnose a failing <code>/verify:run</code> .		Summary of changes for the team.
	<code>/verify:pr</code>		<code>/verify:api</code>
	Full branch review before merging.		API design review after new endpoints.
	<code>/verify:coverage</code>		<code>/verify:performance</code>
	Find test coverage gaps when tests feel thin.		Profile slow queries / suspected N+1.
	<code>/verify:security</code>		<code>/verify:dependencies</code>
	OWASP audit before production.		Outdated & vulnerable packages.
	<code>/verify:system-health</code>		<code>/verify:type-ignores</code>
	Holistic health check before deploy.		Audit type suppressions. Quarterly cleanup.
	<code>/verify:rca</code>		
	Root cause analysis. Required after prod incidents.		

R RELEASE · ship safely
Commit chain first, deploy chain second.

COMMIT CHAIN · IN ORDER

1	<code>/release:commit</code>	Create a git commit with a sensible message.
2	<code>/release:docs</code>	Update README + architecture docs.
	<code>/release:changelog</code>	Generate changelog before a version bump.
	<code>/release:cleanup</code>	Remove dead code + debug prints. Quarterly.

DEPLOY CHAIN · IN ORDER

1	<code>/release:validate</code>	Full validation — tests + Docker + types.
2	<code>/release:env</code>	Audit env vars + secrets before prod.
3	<code>/release:deploy</code>	Pre-deploy checklist — final go / no-go.
	<code>/release:rollback</code>	Reverse the last deploy if things go sideways.

! DEBUG & MONITOR · when things are off
Incidents and continuous health.

	<code>/debug:logs</code>	Parse structured logs. Trace by request ID.
	<code>/monitor:health</code>	Check health endpoints. Post-deploy & continuous.

GITHUB · issues, PRs, sync
`/github:list` → `/github:issue` → `/github:draft` → `/github:sync` before merge.

ISSUES

1	<code>/github:list</code>	Browse open issues by label / assignee.
2	<code>/github:issue</code>	Implement an issue end-to-end — plan, build, verify.
	<code>/github:fix</code>	Fast-path bug fix from an issue ID.
	<code>/github:assign</code>	Assign an issue (yourself or a teammate).
	<code>/github:comment</code>	Post a status / clarifying comment.
	<code>/github:close</code>	Close an issue with a wrap-up note.

PULL REQUESTS & BRANCHES

3	<code>/github:draft</code>	Open a draft PR early for feedback.
4	<code>/github:sync</code>	Sync with main before merge. Every time.
5	<code>/github:pr</code>	Create, update, or merge PRs. <code>--merge</code> to ship.

SKILLS · targeted shortcuts
Call by name when useful.

	<code>/rulecheck</code>	Find & fix CLAUDE.md violations.
	<code>/triage</code>	Label & sort GitHub issues. Weekly.
	<code>/archon</code>	Delegate to remote agents for parallel work.
	<code>/create:diagram</code>	Generate architecture diagrams after planning.

Every command has its place. **Run them in order.**
The right command at the wrong time is still the wrong command.

